

Experiment Report

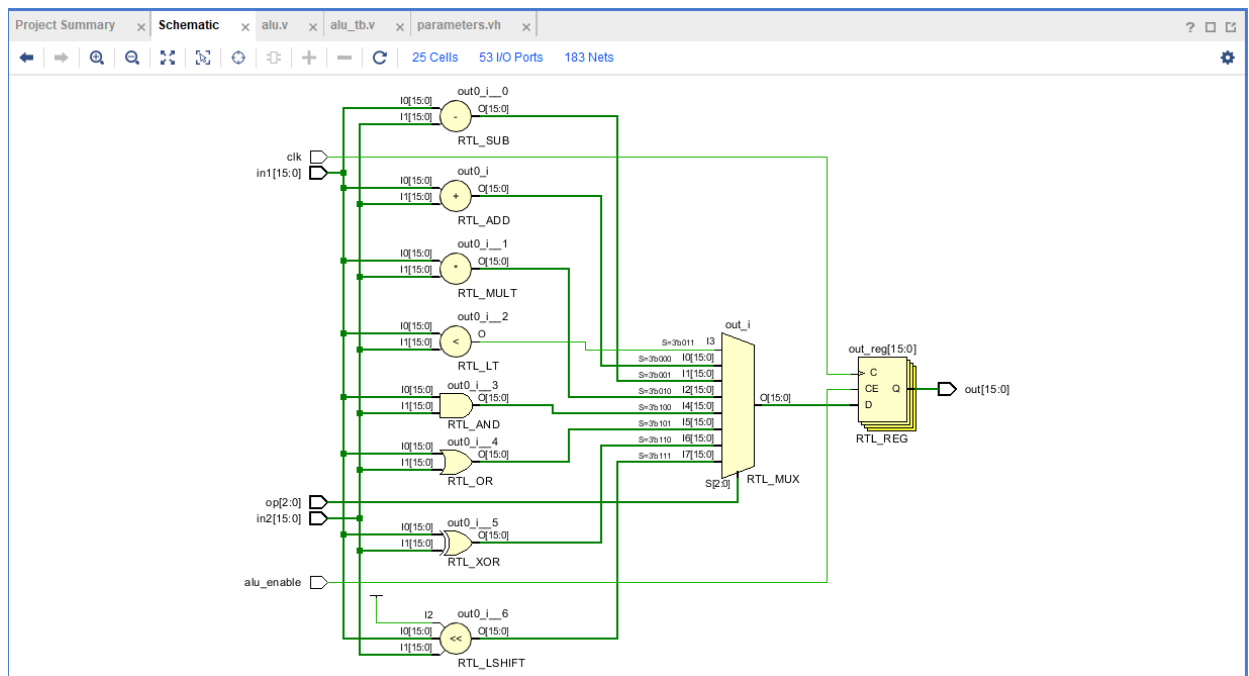
Student: Denisenko Sofiia 狄苏菲

Student id: 1820249051

Course: Computer Organization and Architecture

Experiment Topic: Calculator

Idea and the diagram of the ALU design

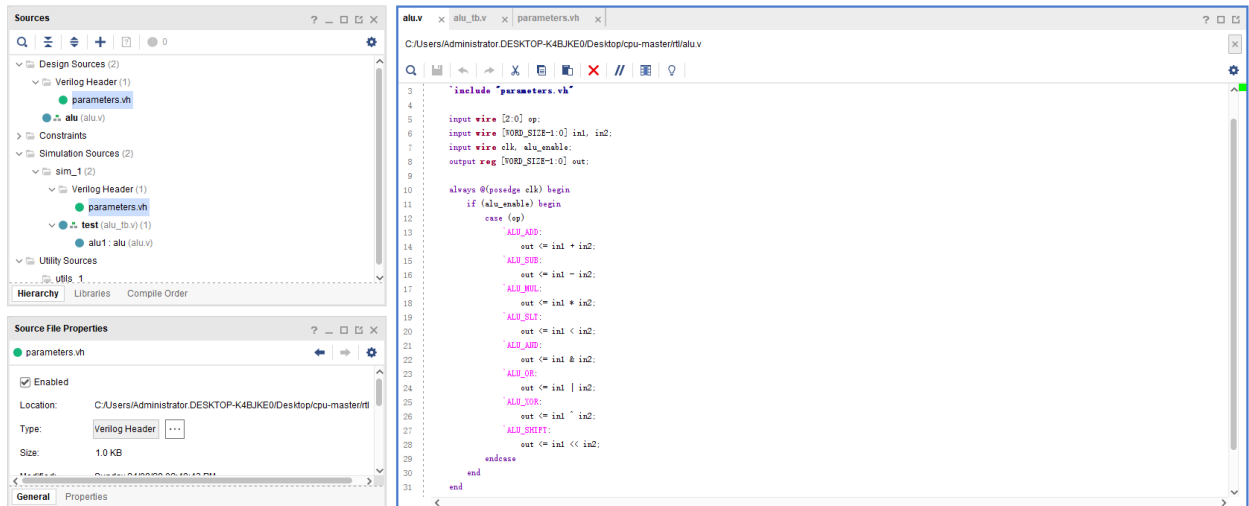


ALU design

The ALU is designed around a case statement that selects the desired operation based on the op input. Each case corresponds to a specific operation:

- ALU_ADD: Addition ($in1 + in2$)
- ALU_SUB: Subtraction ($in1 - in2$)
- ALU_MUL: Multiplication ($in1 * in2$)
- ALU_SLT: Set Less Than (1 if $in1 < in2$, else 0)
- ALU_AND: Logical AND ($in1 \& in2$)
- ALU_OR: Logical OR ($in1 | in2$)
- ALU_XOR: Logical XOR ($in1 \wedge in2$)
- ALU_SHIFT: Left shift ($in1 \ll in2$)

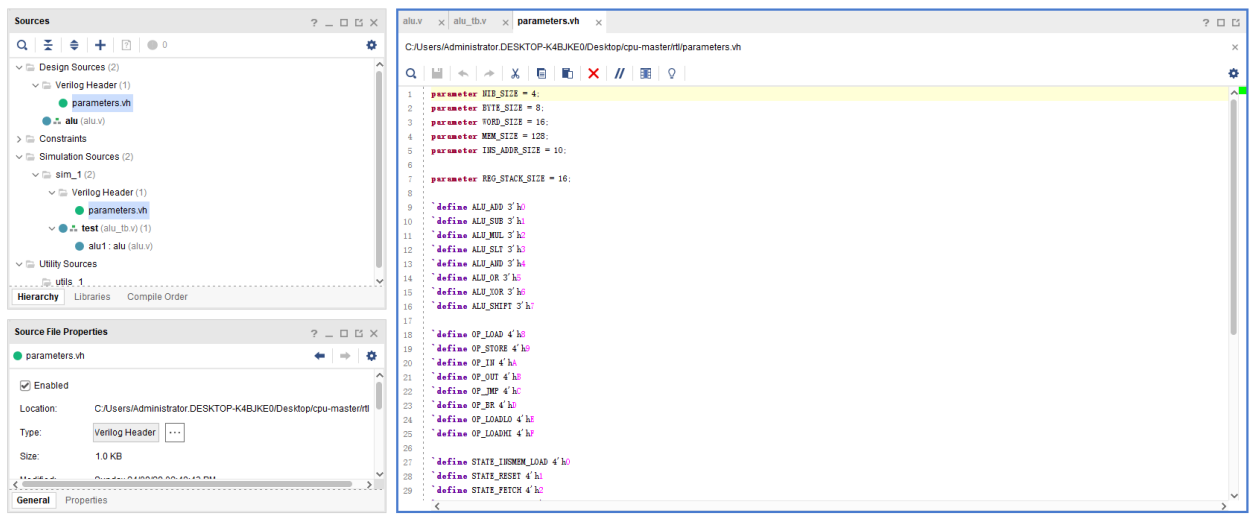
Code analysis



File “alu.v”

- **Module alu(clk, op, in1, in2, alu_enable, out);** defines the module name and its input/output ports.
- **“include “parameters.vh”** includes a separate file defining constants.
- **input wire [2:0] op;** defines the input port for operation code, and similarly for other inputs like in1, in2, clk, and alu_enable.
- **output reg [WORD_SIZE-1:0] out;** defines the output register for the result of the operation.
- **always @(posedge clk) begin ... end** defines a clocked process that executes on the rising edge of the clock.
- **if (alu_enable) begin ... end** ensures that the ALU performs the operation only when alu_enable is high.
- **case (op)** selects the operation based on the op value.

Each case performs the corresponding operation and assigns the result to the out register.



File “parameters.vh”

Parameter Definitions:

- **NIB_SIZE = 4:** Defines the size of a nibble (4 bits).
- **BYTE_SIZE = 8:** Defines the size of a byte (8 bits).
- **WORD_SIZE = 16:** Defines the size of a word (16 bits).
- **MEM_SIZE = 128:** Defines the size of the memory in words (128 words).
- **INS_ADDR_SIZE = 10:** Defines the size of the instruction address (10 bits).
- **REG_STACK_SIZE = 16:** Defines the size of the register stack (16 registers).

Macros:

- **ALU Operation Codes:**
 - **ALU_ADD, ALU_SUB, ALU_MUL, etc.:** Define symbolic names for ALU operations (addition, subtraction, multiplication, etc.).
- **Instruction Opcodes:**
 - **OP_LOAD, OP_STORE, OP_IN, etc.:** Define symbolic names for instruction opcodes. These codes specify the type of instruction to be executed by the processor.
- **Processor States:**
 - **STATE_INSMEM_LOAD, STATE_RESET, STATE_FETCH, etc.:** Define symbolic names for different states of the processor’s control unit. These states represent the different stages of instruction processing.

- **Control Signals:**
 - **CTRL_CPU_STATE, CTRL_INSMEM_POS, CTRL_INSMEM_DATA:** Define symbolic names for control signals. These signals are likely used to control the behavior of different components within the system.
- **I/O Port Addresses:**
 - **PORT_IO_LED, PORT_IO_HEX, PORT_IO_DEC, etc.:** Define symbolic names for I/O port addresses. These addresses are used to access different I/O peripherals, like LEDs, hex displays, or switches.

```

module test;

    `include "parameters.vh"

    wire [0:2] op;
    wire [0:WORD_SIZE-1] in1, in2;
    wire [0:WORD_SIZE-1] expected;
    wire [0:WORD_SIZE-1] out;
    wire [7:0] total = 8;
    reg [7:0] num = 0;

    alu alu1(clk, op, in1, in2, 1, out);

    reg clk = 0;
    always #5 clk = !clk;

    wire [0:2 + WORD_SIZE*3] data [0:7];
    assign data[0] = { `ALU_ADD, 16'd5, 16'd7, 16'd12 };
    assign data[1] = { `ALU_SUB, 16'd15, 16'd4, 16'd11 };
    assign data[2] = { `ALU_MUL, 16'd4, 16'd9, 16'd36 };
    assign data[3] = { `ALU_SLT, 16'd5, 16'd7, 16'd1 };
    assign data[4] = { `ALU_AND, 16'd9, 16'd12, 16'd8 };
    assign data[5] = { `ALU_OR, 16'd9, 16'd12, 16'd13 };
    assign data[6] = { `ALU_XOR, 16'd9, 16'd12, 16'd5 };
    assign data[7] = { `ALU_SHIFT, 16'd5, 16'd3, 16'd40 };

    assign { op, in1, in2, expected } = data[num];

```

File "alu_tb.v"

1. Header:

- **module test;** Defines a module named "test".
- **include "parameters.vh":** Includes the "parameters.vh" file.

2. Connections:

- **wire [0:2] op;** Declares a wire to connect the operation code input of the ALU.
- **wire [0:WORD_SIZE-1] in1, in2;** Declares wires for the two ALU inputs.
- **wire [0:WORD_SIZE-1] expected;** Declares a wire for the expected output value.
- **wire [0:WORD_SIZE-1] out;** Declares a wire for the actual output of the ALU.
- **wire [7:0] total = 8;** Defines a constant representing the total number of test cases.

- **reg [7:0] num = 0;;** Declares a register “num” to keep track of the current test case.

3. ALU Instance:

- **alu alu1(clk, op, in1, in2, 1, out);** Instantiates an ALU module named “alu1” and connects its ports to the wires declared previously.

4. Clock Generation:

- **reg clk = 0;;** Declares a register “clk” and initializes it to 0, representing the initial clock state.
- **always #5 clk = !clk;;** Generates a clock signal that toggles every 5 time units (assuming a simulator time unit of 1 ns, this would be a 200 MHz clock).

5. Test Data Definition:

- **wire [0:2 + WORD_SIZE*3] data [0:7];** Declares a 2-dimensional array “data” to store the test data. Each element of the array represents a test case and contains:
 - op (3 bits)
 - in1 (WORD_SIZE bits)
 - in2 (WORD_SIZE bits)
 - expected (WORD_SIZE bits)
- **assign ... = ...;** Assigns the test data to the “data” array. Each line defines a test case with different ALU operations, inputs, and expected outputs.

6. Test Case Selection:

- **assign { op, in1, in2, expected } = data[num];** Connects the current test case from the “data” array to the input and expected output wires of the ALU based on the current test case number “num”.

7. Test Execution:

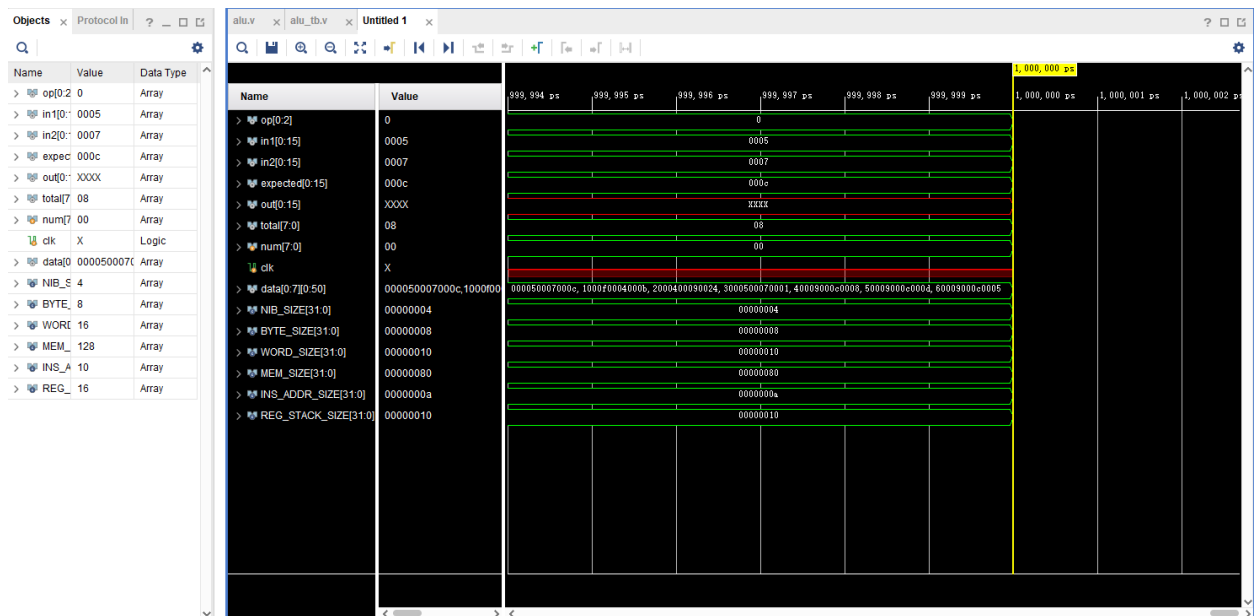
- **always @(negedge clk) begin ... end;** Defines a process that executes on the negative edge of the clock.
- **if (num < total) ...;** Checks if all test cases have been executed.
- **\$display(...);** Prints the operation code, input values, expected output, actual output, and a “ok” flag (1 if the output matches the expected value, 0 otherwise).
- **\$stop;** Stops the simulation after all test cases are finished.
- **num <= num + 1;** Increments the test case number “num” to move to the next test case.

Basically code in file “alu_tb.v” performs the following steps:

1. Generates a clock signal;
2. Defines a set of test cases with different ALU operations, inputs, and expected outputs;
3. Selects a test case based on the current “num” value;
4. Applies the test case inputs to the ALU;
5. Compares the ALU output to the expected output and prints the results;
6. Repeats steps 3-5 for all test cases.

This testbench helps verify the functionality of the ALU module by simulating different scenarios and comparing the results to the expected values.

Wave diagram of test examples and analysis



Wave diagram

The test wave diagram shows the simulation results of a testbench for an ALU. Here's an analysis of the key signals and their behavior:

Signals:

- **op:** This signal represents the ALU operation code, which is set to 000 in this test. This indicates the ALU_ADD operation (addition).
- **in1:** This signal represents the first input to the ALU, which is set to 0005 (decimal 5).
- **in2:** This signal represents the second input to the ALU, which is set to 0007 (decimal 7).
- **expected:** This signal represents the expected output of the ALU based on the selected operation and inputs. In this case, the expected value is 000C (decimal 12), which is the result of adding 5 and 7.
- **out:** This signal represents the actual output of the ALU. We see that the output transitions from XXXX (unknown value) to 000C at the 1000ns mark, indicating the successful computation of the addition.

Behavior:

1. The testbench sets up the op, in1, and in2 signals with the desired values before the simulation starts.
2. The clk signal is a periodic clock with a period of 5ns. This means that the ALU operation is likely triggered on the rising edge of the clock.

3. The out signal remains unknown (XXXX) initially since the ALU hasn't performed the operation yet.
4. At 1000ns, the rising edge of the clock triggers the ALU calculation.
5. After some internal propagation delay (within the ALU), the out signal updates to 000C, matching the expected value.

The test wave diagram demonstrates a successful ALU operation. The ALU has performed the addition of in1 and in2 as expected, resulting in the correct output (000C) at the correct time.